

Contents

1	README	1
2	02-first-login	3
3	03-tui-navigation	5
4	04-roles-and-permissions	7
5	05-logs-and-debugging	8
6	07-lobbies-and-rooms	10
7	08-client-signals	12
8	14-incident-playbooks	14
9	16-training-moderators	16
10	glossary	18

1 README

SekaiLink Core Access

SekaiLink Core Access est le cockpit d'administration de SekaiLink. Il est pense comme un outil TUI Rust lance depuis le bastion Nexus apres une connexion SSH, puis un login SekaiLink/Nexus.

Le nom Core Access evite la confusion avec Client Core et les libretro cores de Sekaiemu.

Objectifs

- Donner aux Admins et au role Service un environnement de travail unique.
- Centraliser logs, lobbies, rooms, releases, packs CDN, bots, scheduler, maintenance, notes et audit.
- Garder Nexus comme source d'identite, de droits, d'audit et de documentation operationnelle.
- Former les nouveaux modérateurs avec des guides imprimables et une reference de commandes complete.

Table des matieres

- [Concepts] (01-concepts.md)
- [Premier login] (02-first-login.md)
- [Navigation TUI] (03-tui-navigation.md)
- [Roles et permissions] (04-roles-and-permissions.md)
- [Logs et debugging] (05-logs-and-debugging.md)
- [Users et configs] (06-users-and-configs.md)
- [Lobbies et rooms] (07-lobbies-and-rooms.md)
- [Signaux clients] (08-client-signals.md)
- [Maintenance mode] (09-maintenance-mode.md)
- [Releases et CDN] (10-releases-and-cdn.md)
- [Scheduler] (11-scheduler.md)
- [Bots Discord/Twitch] (12-bots-discord-twitch.md)
- [Cleanup et backups] (13-cleanup-and-backups.md)
- [Playbooks incidents] (14-incident-playbooks.md)
- [Reference commandes] (15-command-reference.md)
- [Formation modérateurs] (16-training-moderators.md)
- [Runbook Admin] (17-admin-runbook.md)

- [Glossaire](glossary.md)
- [Change control](change-control/README.md)

Regles fortes

- Aucun secret reel dans les docs, PDF ou exemples.
- Toute modification future doit etre documentee dans le meme commit que le code.
- Aucune modification SKLMI sans accord explicite ecrit de Jade.
- Les changements de connectivite Client Core, Sekaiemu ou SKLMI exigent un contrat documente.
- Les actions dangereuses demandent une cible tapee, une raison et un audit.

Generation PDF

Les PDF imprimables sont produits dans `dist/pdf/`:

- `sekailink-core-access-full.pdf`
- `sekailink-core-access-service-training.pdf`
- `sekailink-core-access-command-reference.pdf`
- `sekailink-core-access-incident-playbooks.pdf`
- `sekailink-core-access-quick-reference.pdf`

Commande prevue:

```
```sh
bash docs/sekailink-core-access/scripts/build-pdf.sh
```
```

La pipeline utilise Pandoc avec XeLaTeX. Le script signale clairement les dependances manquantes.

Executable

Le binaire local vit dans `tools/core-access/`. Il lance par default un cockpit terminal plein ecran avec panels, hotkeys, ligne de commande, autocompletion, historique, notes, approvals locales, audit JSONL et dashboard local.

Le shell ligne-par-ligne historique reste disponible avec `--shell`.

Commandes de base:

```
```sh
cargo run --manifest-path tools/core-access/Cargo.toml
cargo run --manifest-path tools/core-access/Cargo.toml -- --shell
cargo run --manifest-path tools/core-access/Cargo.toml -- --command "server status all"
```
```

2 02-first-login

02 - Premier login

Preconditions

- Avoir un compte Linux autorise sur le bastion Nexus.
- Avoir un compte SekaiLink avec role Admin ou Service.
- Avoir un mapping Linux user -> SekaiLink user_id dans Nexus.

Connexion

```
```sh
ssh nexus-vps
sekailink-core-access
```
```

Core Access affiche ensuite un prompt de login SekaiLink.

Verification apres login

Executer:

```
```text
auth whoami
```
```

Resultat attendu:

- Linux user courant.
- SekaiLink user_id.
- Role.
- Capabilities.
- Heure d'expiration de session.

Premier controle

Executer:

```
```text
server status all
release current
maintenance status
```
```

Si un serveur est en rouge, ouvrir ses logs avec F8 ou:

```
```text
server logs <server> <service> --follow
```
```

SSH explicite

Core Access garde l'ouverture SSH interactive derriere une confirmation et une gate locale:

```
```text
ssh open <server> <reason> --confirm ssh:<server>:open
ssh open <server> <reason> --confirm ssh:<server>:open --execute
```
```

Pour executer vraiment, l'environnement local doit contenir:

```
```sh
export SEKAILINK_CORE_ACCESS_SSH_OPEN=1
```
```

Chaque tentative cree un draft audite local avant l'ouverture.

Updates serveurs

Les commandes update ne lancent pas de package manager et ne redemarrent aucun service. Elles servent a creer un plan audite et une trace d'intention:

```
```text
server update plan <server> <scope>
server update apply <server> <plan_id|reason> --confirm server-update:<server>:apply
```
```

Avant une application réelle hors Core Access, faire au minimum:

- `ops snapshot <label>`;
- `server agent-health <server> --execute`;
- `server agent-services <server> --execute`;
- backup des données pertinentes;
- maintenance/broadcast si le scope est visible utilisateur.

3 03-tui-navigation

03 - Navigation TUI

Layout par défaut

```
```text
SekaiLink Core Access
Nexus OK | Link WARN | Worlds OK | Evolution OK | Pulse DOWN
Matrix Live filtered logs Alerts/Jobs
SERVER CPU RAM ERR 22:41 room 38293 joined approvals: 2
Link 38 64 12 22:42 SKLMI reconnect pack check: 14m
... ...
```

```
skl> room logs pas-pour-certo --follow
```

```
```
```

Command line

La ligne de commande est disponible partout.

Fonctions presentes dans le cockpit MVP:

- autocomplete par prefixe avec Tab;
- fleches gauche/droite;
- correction au milieu de ligne;
- historique avec fleche haut/bas;
- Ctrl+A/E;
- Ctrl+W;

Fonctions planifiees:

- selection de suggestion avancee;
- selection visuelle de plage dans les panels logs;
- Alt+B/F;
- reverse search;
- edition multi-line JSON;
- execution non bloquante de flux logs live.

Hotkeys MVP

```
```text
F1 Help
F2 Server status
F3 Command registry
F4 Prepare incident note
F5 Refresh view
F6 Log catalog
F7 Audit search
F8 Prepare log tail
F9 Drop to shell mode
F10 Clear output
F11 Cycle workspace
F12 Panic view
```
```

`Ctrl+Q` ou `Ctrl+C` quitte le cockpit. Le mode `--shell` garde l'ancien prompt ligne-par-ligne pour les sessions de logs longues.

Tabs, splits et workspaces

- F11 change ou sauvegarde le layout.

- Un workspace debug capture les panels ouverts, filtres, pins et notes.
- Les workspaces sont lies a un incident, lobby, room, user ou release.
- Le MVP implemente les workspaces incident via `incident open/status/note/pin/export/close`.

Selection de logs

MVP actuel:

1. Chercher avec `logs search <query> [source|all]`.
2. Isoler avec `logs filter user:<id> lobby:<id> source:<source|all>`.
3. Epingler avec `logs pin <source> <text>`.
4. Ajouter une note avec `logs note <source> <text>`.
5. Exporter avec `logs export [query] --format md`.

Selection visuelle planifiee:

1. Placer le curseur sur le panel log.
2. Selectionner une plage.
3. F4 pour creer une note depuis la selection.
4. F7 pour exporter la selection.

4 04-roles-and-permissions

04 - Roles et permissions

Admin

Admin a le controle complet, avec confirmations et audit:

- reboot/update services;
- publish/rollback release;
- modifier users et roles;
- modifier configs utilisateur;
- give item;
- maintenance full;
- secrets set/rotate;
- approval queue.

Service

Service est quasi-admin live, mais les actions dangereuses demandent approval.

Service peut:

- lire logs, users, configs, lobbies, rooms;
- moderer lobby/chat;
- demander diagnostics client;
- request SKLMI reconnect;
- preparer un give item;
- preparer un rollback;
- preparer une fermeture de room/lobby sensible;
- poster annonces Discord/Twitch selon policy.

Service ne peut pas directement:

- reboot/update serveur;
- publier/rollback release;
- delete user;
- modifier roles;
- modifier secrets;
- modifier DB critique;
- give item sans approval.

Confirmation forte

Toute action dangereuse exige:

- cible tapee exactement;
- raison;
- resume d'impact;
- ops commit.

Exemple:

```
```text
Type target to confirm: link
Reason: emergency restart after stuck release-latest deploy
```
```

5 05-logs-and-debugging

05 - Logs et debugging

Sources

Core Access doit lister deux vues:

- par serveur/service;
- par incident/correlation.

Sources prioritaires:

- journalctl systemd;
- fichiers logs app;
- nginx/CDN;
- Nexus auth/admin;
- Link chat/lobby;
- Worlds generation;
- Evolution packs/releases;
- Pulse;
- Discord social bot;
- Twitch assistant;
- room multiserver logs;
- room events;
- DB/backup logs;
- client reports.

Commandes de base

```
```text
logs list
logs list --by-server
logs list --by-incident
logs tail <source>
logs search <query> [source|all]
logs filter user:<id> lobby:<id> room:<id> correlation:<id> source:<source|all>
logs pin <source> <text>
logs note <source> <text>
logs export [query] --format md
```
```

Etat MVP actuel:

- `logs search` et `logs filter` produisent un plan dry-run par défaut;
- `--execute` reste bloqué sans `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`;
- les prefixes `user:`, `lobby:`, `room:`, `correlation:`, `item:` et `runtime:` sont normalisés en valeurs de recherche;
- `logs pin` écrit une preuve locale dans `log-pins/pins.jsonl`;
- `logs note` écrit une note locale ciblée `log:<source>`;
- `logs export` produit un fichier local Markdown, JSONL ou TXT sous `exports/`;
- les logs client Sekaiemu/SKLMI doivent passer par une demande `client diagnostics-request` avec consentement utilisateur;
- aucune ligne de log n'est écrite dans Nexus DB dans cette tranche.

Notes liées

Une note liée capture:

- extrait original;
- source;
- timestamps;

- tags;
- cible;
- ops_commit_id;
- auteur.

Verbose filtre

Le cockpit doit etre verbose, mais filtre par default:

- erreurs et warnings visibles en haut;
- details disponibles en drilldown;
- bruit groupable par service, room, user ou correlation.

Admin-Agent

Les commandes `server agent-\*` utilisent les admin-agents loopback via SSH:

- `server agent-health [server|all]` prepare `GET /health`;
- `server agent-system [server|all]` prepare `GET /system`;
- `server agent-services [server|all]` prepare `GET /services`;
- `server agent-service <server> <service>` prepare `GET /services/{service}`;
- `server agent-logs <server> <service>` prepare `GET /services/{service}/logs`.

Execution live:

- `--execute` reste bloqué sans `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`;
- les routes protegee demandent le token admin-agent du serveur;
- les tokens sont lus depuis l'environnement local et ne sont jamais imprimes;
- Pulse n'a pas encore de profil admin-agent dans Core Access; utiliser `health probe pulse` en attendant un agent Pulse declare.

Variables attendues:

- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_LINK\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_WORLDS\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_EVOLUTION\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_AGENT\_ADMIN\_TOKEN` comme fallback generique hors Nexus

Controle service:

- `server restart|start|stop` utilise `POST /services/{service}/{action}`;
- role Admin requis;
- `SEKAILINK\_CORE\_ACCESS\_REMOTE\_MUTATION=1` requis;
- confirmation exacte `--confirm <server>:<service>:<action>` requise.

6 07-lobbies-and-rooms

07 - Lobbies et rooms

Lobbies

Commandes:

```
```text
lobby list
lobby open <lobby>
lobby create <lobby_id> <name> [visibility] [owner_username] [description] --confirm lobby:<lobby_id>:create
lobby edit <lobby_id> key=value [key=value...] --confirm lobby:<lobby_id>:edit
lobby close <lobby_id> --confirm lobby:<lobby_id>:close
lobby lock <lobby_id> [reason] --confirm lobby:<lobby_id>:lock
lobby unlock <lobby_id> [reason] --confirm lobby:<lobby_id>:unlock
lobby chat <lobby_id>
lobby join-secret <lobby_id> [reason] --confirm lobby:<lobby_id>:join-secret
lobby broadcast <lobby_id> <message> --confirm lobby:<lobby_id>:broadcast
```
```

`lobby join-secret` est invisible pour les joueurs ordinaires, mais jamais pour l'audit. Il doit apparaitre comme `admin\_observer`.

Etat MVP actuel:

- `lobby list [limit] [query] [visibility] [status] [offset]` prepare un GET prive vers Nexus lobby-admin;
- `lobby open <lobby>` prepare un GET prive vers Nexus lobby-admin;
- `lobby create`, `lobby edit` et `lobby close` utilisent les routes privees Nexus lobby-admin confirmees, avec `--execute`, `SEKAILINK\_CORE\_ACCESS\_NEXUS\_MUTATION=1` et confirmation exacte;
- `lobby lock`, `lobby unlock`, `lobby broadcast` et `lobby join-secret` creent des drafts audites locaux;
- `lobby chat` prepare une recherche de logs sur `link:chat-api`;
- l'execution live est read-only, bloquee par default, et exige `--execute`, `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1` et `SEKAILINK\_CORE\_ACCESS\_NEXUS\_LOBBY\_ADMIN\_TOKEN`;
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ADMIN\_TOKEN` reste un fallback local compatible;
- les mutations lobby live sont bloquees sans role/gate/confirmation.

Rooms

Commandes:

```
```text
room list [limit] [query] [room_type] [connection_state] [offset]
room open <room_id>
room summary <room_id>
room events <room_id> [limit] [event_type] [severity] [offset] [source]
room logs <room_id> --follow
room snapshot <room_id>
room sync <room_id> [reason] --confirm room:<room_id>:sync
room pending-items <room_id>
room client-reports <room_id> [limit] [report_type] [severity] [source] [offset]
room request-skmi-reconnect <room_id> <player> [reason] --confirm room:<room_id>:request-skmi-reconnect
room disconnect-runtime <room_id> <player|runtime> [reason] --confirm room:<room_id>:disconnect-runtime
room give-item <room_id> <target> <item> [reason] --confirm room:<room_id>:give-item
```
```

Etat actuel:

- `room list` utilise `GET /rooms` sur Nexus room-query;

- `room open` et `room snapshot` utilisent `GET /rooms/{room\_id}`;
- `room summary` utilise `GET /rooms/{room\_id}/diagnostics`;
- `room events` utilise `GET /rooms/{room\_id}/events`;
- `room client-reports` utilise `GET /rooms/{room\_id}/client-reports`;
- `room pending-items` lit le snapshot et demande d'inspecter
`pending\_delivery\_count`/`received\_items`;
- `room logs` prepare une recherche `link:room-runtime` ou un follow
`sekailink-room-server`;
- `room sync`, `room request-sklmi-reconnect`, `room disconnect-runtime` et
`room give-item` creent des drafts audites locaux, sans mutation runtime.

Execution read-only room-query:

- `--execute`
- `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ROOM\_QUERY\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ADMIN\_TOKEN` fallback

Give item

Policy:

- Admin ou approval Admin.
- Snapshot automatique avant action.
- Raison obligatoire.
- Impact live affiche.
- Ops commit.

SKLMI reconnect

`room request-sklmi-reconnect` envoie un signal client/runtime. Une modification SKLMI pour supporter une nouvelle forme de reconnect demande accord explicite.

7 08-client-signals

08 - Signaux clients

Les signaux clients sont realtime. Le fallback poll peut exister pour robustesse, mais le cockpit doit viser l'immediat.

Signaux

```
```text
client broadcast
client maintenance-banner
client force-update-prompt
client request-relogin
client refresh-lobby
client refresh-room
client request-sklmi-reconnect
client restart-runtime
client clear-cache-request
client diagnostics-request
client diagnostics-list
client diagnostics-export
```
```

Diagnostics

Regles:

- consentement utilisateur obligatoire;
- logs et configs redacted;
- upload lie a un incident ou ops commit;
- jamais de secret client brut.

Etat Core Access actuel:

- `client diagnostics-request` cree une demande locale `draft-consent-required`;
- `client broadcast`, `client maintenance-banner`, `client force-update-prompt`, `client request-relogin`, `client refresh-lobby`, `client refresh-room`, `client request-sklmi-reconnect`, `client restart-runtime` et `client clear-cache-request` creent des drafts locaux audites dans `drafts/client-signal.jsonl`;
- les signaux sensibles exigent `--confirm client:<action>:<target>`;
- `client refresh-lobby` et `client refresh-room` sont des drafts sans confirmation forte, car ils ne sont pas destructifs;
- `client clear-cache-request` reste marque approval-required dans le detail du draft;
- aucun signal client n'est envoye par Core Access actuellement;
- aucun fichier Client Core, Sekaiemu ou SKLMI n'est collecte par Core Access;
- `client diagnostics-list` liste les demandes locales;
- `client diagnostics-export` exporte les demandes et le contrat de bundle attendu.

Bundle attendu quand l'integration client sera implementee:

- `client-core/main.log`, `client-core/renderer.log`, `client-core/updater.log`;
- `sekaiemu/sekaiemu.log`, `sekaiemu/stdout.log`, `sekaiemu/stderr.log`;
- `sklmi/sklmi.log`, `sklmi/stdout.log`, `sklmi/stderr.log`;
- configs redacted et contexte de lancement;
- informations systeme sans secrets.

Toute modification SKLMI necessaire pour produire ces fichiers demande accord explicite ecrit de Jade avant implementation.

Broadcast

Ciblage:

- global;
- server;
- lobby;
- room;
- role;
- version;
- game.

Exemple:

```
```text  
broadcast lobby pas-pour-certo "Maintenance de room dans 5 minutes." --confirm broadcast:lobby:pas-pour-certo
```
```

Etat actuel: les commandes `broadcast ...` creent des drafts locaux audites dans
`drafts/broadcast.jsonl`. Aucun message n'est envoye aux clients, rooms,
lobbies, Discord ou Twitch tant que le transport serveur n'est pas connecte.

8 14-incident-playbooks

14 - Playbooks incidents

Room server down

```
1. `incident open room-<room> sev2 room server symptoms`
2. `room list`
3. `room summary <room>`
4. `room logs <room> --follow`
5. `server status link`
6. `logs filter room:<room> source:link:room-runtime`
7. `incident pin room-<room> link:room-runtime <important line>`
8. `incident note room-<room> <decision or next step>`
9. Si necessaire: `ops snapshot room-<room>`.
```

SKLMI non connecte

```
1. `incident open sklmi-<room>-<player> sev3 SKLMI not connected`
2. `room summary <room>`
3. Verifier presence player et runtime.
4. `room events <room>`
5. `logs filter runtime:<player> source:link:room-runtime`
6. `incident pin sklmi-<room>-<player> link:room-runtime <important line>`
7. `client request-sklmi-reconnect <player>`
8. Si echec: `client diagnostics-request <player> sklmi-<room>-<player> <reason> --include sekaiemu,sklmi,client`
9. Exporter la demande si besoin avec `client diagnostics-export <player> --file diagnostics-<player>.md`.
```

Client update brise

```
1. `release current`
2. `release verify-cdn`
3. `logs filter release:<version>`
4. `maintenance enable client_update_only --message "...`
5. `release rollback <version>` si Admin.
```

Generation bloquee

```
1. `server status worlds`
2. `logs tail worlds:generation`
3. `schedule history health-probe-worlds`
4. `maintenance enable generation_only --message "...`
```

CDN pack invalid

```
1. `pack repo list`
2. `pack validate <id>`
3. `pack rollback <id> <version>`
4. `broadcast game <game> "...`
```

Bot Discord/Twitch down

```
1. `discord status` ou `twitch status`
2. `discord logs` ou `twitch logs`
3. `discord reload` ou `twitch connect <channel>`
4. Creer note incident si recurring.
```

Maintenance d'urgence

```
1. F12 Panic.
2. Lire impact.
3. `incident open emergency-<scope> sev1 <summary>`
4. Choisir scope minimal.
```

5. Taper cible + raison.
6. Broadcast.
7. Collect logs bundle.
8. ``incident export emergency-<scope> --file emergency-<scope>.md``

Releve apres incident

1. ``ops timeline <label>``
2. ``incident status <label>``
3. ``incident export <label> --file <label>.md``
4. ``ops handoff shift-<date> --file shift-<date>.md``

``incident export`` isole un incident. ``ops handoff`` resume le shift complet: incidents, notes, pins, approvals, maintenance, scheduler, packs, banners et audit local.

9 16-training-moderators

16 - Formation modérateurs

Objectif

Former le role Service pour aider en live sans mettre la production en danger.

Parcours

1. Lire Concepts.
2. Lire Premier login.
3. Lire Navigation TUI.
4. Lire Roles et permissions.
5. Faire les exercices logs.
6. Faire les exercices lobbies/rooms.
7. Faire un incident simule.
8. Lire les limites Service.

Exercices

Exercice 0 - Verifier le poste Core Access

```
```text
auth whoami
ops doctor
ops paths
ops exports
```
```

But: confirmer le role, l'etat local du cockpit, les chemins de travail, et les exports deja presents avant un shift.

Exercice 1 - Diagnostiquer un joueur

```
```text
user search <name>
user open <user>
user sessions <user>
user configs <user>
```
```

But: trouver si le joueur est connecte, sur quel device et avec quelle config.

Exercice 2 - Suivre une room

```
```text
room summary <room>
room events <room>
room logs <room> --follow
```
```

But: identifier qui est connecte, quel runtime parle, et s'il y a des pending items.

Exercice 3 - Creer un incident local

```
```text
incident open training-room sev4 training incident
incident note training-room player reported a reconnect issue
incident pin training-room link:room-runtime sample runtime line
incident status training-room
incident export training-room --file training-room.md
```
```

incident close training-room resolved during training
```

But: comprendre le journal append-only, les notes, les pins et l'export  
Markdown sans toucher a la production.

### Exercice 4 - Faire une releve de shift

``text  
ops timeline training-room  
ops handoff training-shift --file training-shift.md  
```

But: produire un rapport de releve lisible pour un autre modérateur.

A ne jamais faire sans Admin

- publier une release;
- rollback release;
- reboot/update serveur;
- modifier role;
- modifier secret;
- injecter item;
- modifier DB critique;
- ignorer un backup gate.

Checklist avant premier shift

- Je sais me connecter au bastion.
- Je sais verifier mon role.
- Je sais lancer `ops doctor` avant un shift.
- Je sais lister logs et rooms.
- Je sais creer un incident local, une note et un pin.
- Je sais produire une releve de shift.
- Je sais demander approval.
- Je sais utiliser F12 Panic en lecture avant action.

10 glossary

Glossaire

****Admin****: role complet avec actions dangereuses, approvals et maintenance.

****Approval queue****: file de demandes preparees par Service et approuvees par Admin.

****Bastion Nexus****: serveur Nexus utilise comme point d'entree SSH et hote de Core Access.

****Client Core****: app desktop SekaiLink utilisateur.

****Core Access****: cockpit TUI admin. Ne pas confondre avec Client Core ou les libretro cores.

****Evolution****: role distribution, CDN, assets, packs et releases.

****Link****: API publique, lobbies, chat, room passthrough, release-latest.

****Nexus****: identite, users, configs, audit, scheduler et source de confiance.

****Ops commit****: entree atomique d'audit pour une action.

****Pulse****: assistant/config helper.

****Room****: session runtime/live associee a une sync.

****Sekaitemu****: emulator renderer/runtime visuel.

****Service****: role support/moderation puissant, avec approvals pour actions dangereuses.

****SKLMI****: companion tracker/runtime qui possede la logique tracker.

****Worlds****: generation service.